

Assignment 3: Understanding Algorithm Efficiency and Scalability

Introduction

This assignment examines the efficiency and scalability of two algorithms: Randomized Quicksort and Hashing with Chaining. The work includes implementing both algorithms, analyzing their theoretical time complexity, and comparing their performance under different input conditions.

The first part compares randomized Quicksort with deterministic Quicksort, which selects the first element as its pivot. The second part implements a hash table using separate chaining and analyzes how collisions and the load factor affect performance.

Part 1: Randomized Quicksort Analysis

1. Implementation

Randomized Quicksort is a divide-and-conquer sorting algorithm. It selects a pivot uniformly at random from the current subarray, partitions the elements around the pivot, and recursively processes the resulting partitions.

The implementation uses an iterative stack to avoid excessive recursion depth. It also handles empty arrays, single-element arrays, sorted arrays, reverse-sorted arrays, and arrays containing repeated values.

The deterministic implementation uses the first element of each subarray as its pivot. Both algorithms return a sorted copy of the input array.

2. Average-Case Time Complexity Analysis

Best, average, and worst cases

The best-case and expected average-case time complexities of randomized Quicksort are both $O(n \log n)$. Its worst-case time complexity is $O(n^2)$.

The best case occurs when the pivot repeatedly divides the array into approximately equal partitions. The worst case occurs when the pivot repeatedly produces highly unbalanced partitions, such as one empty partition and one partition containing nearly all remaining elements.

Recurrence relation

For an array of size n , partitioning requires $O(n)$ time. If the pivot ends up at position k , the recursive work is performed on subarrays of sizes k and $n - k - 1$.

The recurrence is:

$$T(n) = T(k) + T(n - k - 1) + \Theta(n)$$

For randomized Quicksort, the pivot position is uniformly distributed among the n possible positions. Therefore, the expected running time satisfies:

$$E[T(n)] = (1/n) \sum E[T(k) + T(n - k - 1)] + \Theta(n)$$

for k from 0 to $n - 1$.

By symmetry, this becomes:

$$E[T(n)] = (2/n) \sum E[T(k)] + \Theta(n)$$

for k from 0 to $n - 1$.

Solving this recurrence gives:

$$E[T(n)] = \Theta(n \log n)$$

The random pivot makes consistently poor partitions unlikely over the full sequence of partitioning steps. Although an individual pivot can produce an unbalanced partition, the expected total work remains $O(n \log n)$.

Indicator random variable interpretation

Another way to analyze randomized Quicksort is to count comparisons.

Let X_{ij} be an indicator variable equal to 1 if the elements with sorted ranks i and j are compared, and 0 otherwise.

Two elements are compared only if one of them becomes a pivot before any element whose sorted rank lies between them. The probability of this event is:

$$P(X_{ij} = 1) = 2 / (j - i + 1)$$

for $i < j$.

The total number of comparisons is:

$$C = \sum X_i$$

Using linearity of expectation:

$$E[C] = \sum E[X_i]$$

The resulting sum is bounded by a constant multiple of $n \log n$, establishing the expected $O(n \log n)$ comparison complexity.

3. Empirical Comparison

The benchmark compares randomized Quicksort with deterministic first-element-pivot Quicksort using the following input distributions:

- Randomly generated arrays
- Already sorted arrays
- Reverse-sorted arrays
- Arrays containing repeated values

Input sizes of 100, 500, and 1,000 elements are tested. Each combination is measured three times, and the mean, minimum, and maximum runtimes are recorded in milliseconds.

Theoretical expectations

Input distribution	Deterministic first-pivot Quicksort	Randomized Quicksort
Random	Expected $\Theta(n \log n)$	Expected $\Theta(n \log n)$
Sorted	$\Theta(n^2)$	Expected $\Theta(n \log n)$
Reverse-sorted	$\Theta(n^2)$	Expected $\Theta(n \log n)$
All repeated values	$\Theta(n^2)$ with this two-way partition	$\Theta(n^2)$ with this two-way partition

The repeated-value case is important. Random pivot selection alone does not guarantee efficient handling of duplicate-heavy arrays. With the two-way partition used in this implementation, an array containing identical values can still produce highly unbalanced partitions.

A three-way partitioning approach, which groups values smaller than, equal to, and greater than the pivot, can improve performance on arrays with many repeated values.

Results and discussion

The benchmark script generates the actual runtime measurements and saves them in `benchmark_results.csv`.

The theoretical analysis predicts that deterministic Quicksort will perform poorly on already sorted and reverse-sorted inputs because selecting the first element repeatedly produces highly unbalanced partitions. Randomized Quicksort avoids consistently selecting an extreme pivot by choosing a pivot uniformly at random.

For random input, both algorithms are expected to have average-case $O(n \log n)$ performance, although their measured runtimes may differ due to implementation overhead and pivot selection.

For arrays containing identical values, both implementations can exhibit quadratic behavior because the partitioning method places equal elements on the same side of the pivot.

Differences between measured and theoretical performance can result from random variation, interpreter overhead, system load, and the relatively small input sizes used in the experiment.

Part 2: Hashing with Chaining

1. Implementation

A hash table stores key-value pairs and uses a hash function to map each key to a bucket index.

When two keys map to the same bucket, a collision occurs. Separate chaining resolves collisions by storing multiple key-value pairs in a list associated with each bucket.

The implementation supports:

- **Insert:** Adds a new key-value pair or updates an existing key.
- **Search:** Returns the value associated with a key, or `None` if the key is absent.
- **Delete:** Removes a key-value pair and returns whether the deletion succeeded.

The table uses a universal-family-style hash function for integer keys:

$$h(k) = ((a \times k + b) \bmod p) \bmod m$$

where:

- p is a prime number.
- a is a randomly selected nonzero multiplier.
- b is a randomly selected offset.
- m is the current number of buckets.

When the load factor exceeds the configured threshold of 0.75, the table doubles its capacity and redistributes its existing entries.

2. Expected Time Complexity

Simple uniform hashing

Under simple uniform hashing, each key is assumed to have an equal probability of being assigned to any bucket, independently of other keys.

Let:

$$\alpha = n / m$$

where n is the number of stored keys and m is the number of buckets.

The value α is called the **load factor**. It represents the average number of elements per bucket.

Under simple uniform hashing, the expected number of elements in a chain is α .

Search

An unsuccessful search examines the chain corresponding to the requested bucket. Its expected time complexity is:

$$O(1 + \alpha)$$

A successful search also takes expected $O(1 + \alpha)$ time under the usual uniform-hashing assumptions.

Insert

Insertion first checks whether the key already exists. The expected cost is $O(1 + \alpha)$ when updating or checking for an existing key.

Adding a new entry to the end of a chain has constant insertion cost once the bucket is located. Therefore, the expected insertion cost is $O(1 + \alpha)$ when duplicate-key checking is included.

When resizing occurs, rebuilding the table takes $O(n)$ time. However, if the table doubles only when the load factor exceeds a fixed threshold, the amortized insertion cost remains expected $O(1)$ under the assumption of a bounded load factor and suitable hashing.

Delete

Deletion searches the chain for the key and removes the matching entry.

Its expected time complexity is:

$$O(1 + \alpha)$$

This assumes that chain traversal and removal take time proportional to the chain length.

Complexity summary

Operation	Expected time
Search	$O(1 + \alpha)$
Insert	$O(1 + \alpha)$, excluding resizing
Delete	$O(1 + \alpha)$
Resize	$O(n)$
Amortized insertion with doubling	Expected $O(1)$ when α is kept bounded

3. Effect of the Load Factor

The load factor measures how full the hash table is relative to its number of buckets.

A low load factor means that each bucket contains relatively few elements. This reduces the expected number of comparisons required to search, insert, or delete a key.

A high load factor means that chains tend to become longer, increasing the amount of work required to locate an element.

For example, if a table contains 100 elements and has 200 buckets, its load factor is 0.5. If the same table has only 50 buckets, its load factor is 2.0.

Under simple uniform hashing, the expected chain length is proportional to the load factor.

4. Strategies for Minimizing Collisions

Several strategies can improve hash table performance:

1. **Dynamic resizing:** Increase the number of buckets when the load factor exceeds a threshold.
2. **Suitable hash functions:** Use a hash function that distributes keys evenly across the available buckets.
3. **Universal hashing:** Randomly select a hash function from a suitable family to reduce the likelihood of persistent collision patterns.
4. **Load-factor management:** Maintain a reasonable ratio between stored elements and bucket count.

The implementation uses dynamic resizing by doubling the number of buckets when the load factor exceeds 0.75.

Resizing requires rehashing the existing entries because their bucket indices depend on the new capacity.

Conclusion

This assignment demonstrates how algorithm design choices affect efficiency and scalability.

Randomized Quicksort achieves expected $O(n \log n)$ time by choosing pivots uniformly at random, while deterministic first-element-pivot Quicksort can degrade to $O(n^2)$ on sorted and reverse-sorted arrays.

Hashing with chaining supports expected $O(1 + \alpha)$ search, insertion, and deletion under simple uniform hashing. Managing the load factor through dynamic resizing helps maintain efficient operations as the table grows.

The combination of theoretical analysis and empirical benchmarking illustrates why input distribution, pivot selection, collision resolution, and data structure design are important when evaluating algorithm performance.